

课程笔记

Codex 上下文压缩 Compact 与 Handoff

基于公开课程资料整理

2025

Codex 上下文压缩 Compact Handoff

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 15 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么需要上下文压缩？	2
2 Compact 的基本原理	2
2.1 触发方式	2
2.2 先压缩，后交接	2
2.3 本章小结	2
3 OpenAI Provider 与非 OpenAI Provider 的路径分叉	2
3.1 两条不同的代码路径	2
3.2 本章小结	3
4 Compact Prompt 与 Handoff Prompt 的内容	3
4.1 压缩提示词 (Compact Prompt)	3
4.2 交接提示词 (Handoff Prompt / Summary Prefix)	3
4.3 完整的两步流程	3
4.4 本章小结	4
5 破译远端 Compact 的 System Prompt	4
5.1 Prompt Injection 实验	4
5.2 不同模型的实验结果	4
5.3 本章小结	5
6 总结与延伸	5
6.1 拓展阅读	5

1 引言：为什么需要上下文压缩？

在使用 Codex 等 Coding Agent 进行长时间开发时，对话的上下文会不断累积。当上下文长度逼近模型的 Context Window 上限时，如果不加处理，Agent 将无法继续接收新信息。**上下文压缩 (Compact)** 正是为了解决这一问题而设计的机制——在保留关键信息的前提下，将冗长的对话历史压缩为精炼的摘要，使 Agent 能够“无缝衔接”后续工作。

本期主要参考了 Twitter 上一篇知名帖子对 Codex Compact 机制的逆向分析，结合 Codex 源码进行解读。

核心问题

Codex 的上下文压缩 (Compact) 是如何工作的？它背后的 System Prompt 是什么？压缩后如何交接 (Handoff) 给下一个 LM？

2 Compact 的基本原理

2.1 触发方式

Compact 可以通过两种方式触发：

- **手动触发**：用户主动执行压缩命令。
- **自动触发**：当上下文长度达到 Context Window 的 **90%** 时，系统自动执行一次压缩。

2.2 先压缩，后交接

无论哪种触发方式，Compact 的流程都是一致的：

两步走：Compact + Handoff

1. **压缩 (Compact)**：将当前全部对话历史压缩为一份精炼的“交接摘要”。
2. **交接 (Handoff)**：将压缩后的摘要注入到新的对话上下文中，使“接手”的 LM 能够继续工作。

2.3 本章小结

Compact 的本质是一个“上下文检查点” (Context Checkpoint)，在不丢失关键信息的前提下为 Agent 腾出 Context Window 空间。

3 OpenAI Provider 与非 OpenAI Provider 的路径分叉

3.1 两条不同的代码路径

Codex 源码中根据 Provider 类型走不同的 Compact 路径：

- **OpenAI Provider**：调用专门的 `response.compact` API——这是一个专用的压缩接口，只需提交对话历史，即可获得压缩版内容。System Prompt 和 Handoff Prompt 在**远端** (OpenAI 服务器端) 维护。

- **非 OpenAI Provider**: 调用普通的 `response.create` 接口, 在**本地**注入 Compact Prompt 和 Handoff Prompt。

response.compact API

这是 OpenAI 提供的专用压缩接口。调用方只需提交对话历史 (messages), API 返回压缩后的摘要。无需在客户端维护压缩提示词。

3.2 本章小结

Provider 的不同决定了压缩提示词的维护位置: OpenAI 走远端 `response.compact`, 其余走本地 `response.create` 并显式注入提示词。

4 Compact Prompt 与 Handoff Prompt 的内容

4.1 压缩提示词 (Compact Prompt)

Codex 源码中为非 OpenAI Provider 提供了明文的压缩提示词 (`compact.prompt.md`), 核心指令如下:

Compact Prompt 全文要义

”你正在执行一个上下文检查点的压缩。请为另一个将接手该任务的 LM 创建一份交接摘要, 包含:

- 当前的进度
- 已经做出的关键决策
- 重要的上下文、约束条件和用户偏好
- 尚未完成的工作
- 继续工作所需的任何关键数据、示例或资料

一定要简洁、结构化, 并专注于帮助下一个 LM 无缝衔接工作。”

4.2 交接提示词 (Handoff Prompt / Summary Prefix)

交接发生在压缩完成之后, `summary.prefix.md` 的内容如下:

Handoff Prompt 全文要义

”另外一个语言模型已经开始解决这个问题, 并生成了其思考过程的摘要。你也可以访问该语言模型所使用的工具状态。请利用这些信息在已有工作的基础上继续进行, 避免重复劳动。以下就是该语言模型的摘要——请利用摘要中的信息辅助你自己的分析。”

4.3 完整的两步流程

1. Step 1 —压缩:

- 输入：System Prompt + Compact Prompt + 全部对话历史。
- 输出：一份结构化的”密文”（压缩摘要）。

2. Step 2 —交接：

- 输入：System Prompt + Handoff Prompt + 压缩摘要 + 用户的新请求。
- 输出：正常的 Agent 响应，无缝继续工作。

4.4 本章小结

Compact Prompt 指导模型生成”交接摘要”，Handoff Prompt 指导接手模型利用摘要继续工作。两者配合实现了”压缩 → 交接 → 继续”的闭环。

5 破译远端 Compact 的 System Prompt

5.1 Prompt Injection 实验

Twitter 帖子的作者通过 Prompt Injection 技术，尝试破译 OpenAI 远端 `response.compact` API 背后的 System Prompt。实验方法如下：

1. 在要压缩的内容中植入一段特殊指令（Injection）。
2. 指令要求模型在产生压缩摘要之前，先**完整引用**收到的所有涉及”context checkpoint””handoff summary””concise”等关键词的指令原文。
3. 将植入的指令伪装为”强制性质量保证步骤”。

Prompt Injection 的伦理边界

此实验仅用于技术研究目的，帮助社区理解 Codex 的内部机制。在生产环境中，Prompt Injection 是一种安全风险，应当被防御而非利用。

5.2 不同模型的实验结果

- **GPT-4.1**：直接输出了 Compact 的 System Prompt 原文（与源码中的 `compact.prompt.md` 一致），但未输出 Handoff Prompt。
- **GPT-5.2**：虽然声称”不可以提供完整内容”，但事实上**同时**输出了 Compact System Prompt 和 Handoff Prompt 的原文。
- **GPT-5.3 (Codex)**：**拒绝回答**，成功抵御了 Prompt Injection。

关键结论

远端 `response.compact` API 使用的 System Prompt 和 Handoff Prompt，与 Codex 源码中为非 OpenAI Provider 提供的明文版本**基本一致**。换言之，OpenAI 并未在远端使用更复杂的压缩策略。

5.3 本章小结

通过 Prompt Injection 实验证实，远端和本地的压缩提示词本质相同。同时可以观察到，更新的模型（GPT-5.3）对 Injection 攻击的防御能力显著增强。

6 总结与延伸

本期深入分析了 Codex 的上下文压缩（Compact）与交接（Handoff）机制。核心要点回顾：

1. **触发条件**：手动或自动（Context Window 达到 90%）。
2. **两步流程**：先用 Compact Prompt 生成结构化交接摘要，再用 Handoff Prompt 注入新对话上下文。
3. **Provider 分叉**：OpenAI 走远端 `response.compact` API，非 OpenAI 走本地 `response.create` + 显式注入提示词。
4. **远端与本地一致**：Prompt Injection 实验证实远端的压缩/交接提示词与源码中的明文版本基本相同。
5. **模型防御能力演进**：GPT-4.1 易被注入，GPT-5.2 部分泄露，GPT-5.3 成功抵御。

Compact 机制的设计体现了一个重要的工程思想：**在有限的 Context Window 约束下，通过结构化的摘要生成与交接协议，实现长时间任务的连续性**。这不仅适用于 Coding Agent，也为任何需要长期对话管理的 AI 系统提供了参考范式。

6.1 拓展阅读

- Codex 开源代码仓库（Compact 相关模块）
- OpenAI `response.compact` API 文档
- 相关 Twitter 帖子对 Compact System Prompt 的逆向分析